

Assignment 2 DA3408

Sidhartha Durgam

September 2026

Question 1 — Single-Stage vs. Multi-Stage Docker

1. Naive Build & Endpoints

The naive, single-stage Dockerfile successfully built and ran the API. The container correctly exposed the Swagger UI, serving the `/healthz` endpoint (returning a 200 OK status) and the `/predict` endpoint, which successfully classified test strings like "WIN a FREE iPhone now!" as `spam`. The resulting image size for the `spam-api-naive:latest` build is **2.28GB**.

2. Multi-Stage Build & Size Reduction

The multi-stage Dockerfile successfully built and exhibited identical behavior to the naive version, accurately classifying "Hi, My name is Sidhartha Durgam." as `ham`. The new resulting image size for `spam-api-multi:latest` is **153MB**. This represents a massive size reduction of approximately **93.4%**.

Note: Visual evidence for execution, endpoint validation, and reported image sizes can be found in the GitHub repository under the `q1_*` image files.

3. Specific Explanation of Size Reduction

The multi-stage image is smaller because the final runtime stage uses the much smaller `python:3.14-slim` base image instead of the full `python:3.14` image used by the naive build. The builder stage contains the full Python environment and any build dependencies required to install the packages, but these are not carried into the final image; only the installed packages from `/build/deps` and the application files are copied into the slim runtime image.

Question 2 — Multi-Container Orchestration with Docker Compose

1. Caching Implementation & Accuracy

The API was updated to implement Redis-based memoization. Before running a prediction, the endpoint checks Redis for the exact input text. On a cache MISS, the API computes the prediction, stores it in Redis with a designated TTL, and returns the result. On a cache HIT, it instantly returns the cached label. Both scenarios successfully and accurately return the correct predictions.

2. Docker Compose & Inter-Service Networking

The two-container stack is defined in the `docker-compose.yml` file (available in the repository). It builds the `api` service using the optimized multi-stage Dockerfile and pulls the official `redis:7-alpine` image for the `cache` service. Compose automatically provides DNS resolution, allowing the API to connect to the Redis database using the service name `cache` directly as its hostname.

3. Measurable Cache-Hit Speedup

Executing a timing test proved the caching layer's effectiveness. The initial request for the text string "I'm Sidhartha Durgam" triggered a `cache_miss` and took **14.25ms** to complete due to the model's inference overhead. Sending the exact same request immediately after triggered a `cache_hit`, returning the label in just **0.36ms**. This measurable speedup confirms the cache successfully bypasses the computational bottleneck.

Note: Visual timing evidence (`q2_cache_miss.png` and `q2_cache_hit.png`) and the `docker-compose.yml` configuration are available in the GitHub repository.

4. Docker Compose vs. Kubernetes

Docker Compose solves the problem of defining, networking, and running multiple dependent containers (like an API and a database) simultaneously on a **single host**. It eliminates the need for manual `docker run` commands and network bridging. Kubernetes differs in scope by providing **cluster-scale orchestration**; it distributes and manages containers across a fleet of multiple nodes, offering advanced self-healing, load balancing, and scaling capabilities that a single-host Compose setup cannot achieve.

Question 3 — Kubernetes Indexed Job: Parallel Data Validation

1. Parallelism Choice & Manifest

With 2 nodes of 2 CPUs each, there are 4 total allocatable CPUs. I chose `parallelism: 4` and requested 1 CPU per pod to ensure exactly 4 pods run concurrently (2 on each node), maximizing hardware utilization without leaving excess pods indefinitely `Pending`. The complete configured manifest is available in the repository as `job-validator.yaml`.

2 & 3. API Data Collection, Storage Justification & Execution Evidence

Minikube uses node-local storage, meaning a shared volume provisioned on node 1 cannot be reliably read by a pod on node 2. Therefore, aggregating logs via the Kubernetes Python API is the safest, most reliable method for cross-node collection. The Kubernetes Python API was used to read each pod's logs and collect the result for all eight Indexed Job completions. The invalid-row counts were 11, 9, 9, 8, 10, 12, 5, and 13 for completion indexes 0–7 respectively, giving a deterministic total of 77 invalid rows. The execution evidence also shows four pods running concurrently across the two nodes.

Note: Visual terminal execution evidence (`q3-execution-evidence.png`) and the final script output table (`q3-api-results.png`) can be found in the GitHub repository.

4. Theoretical 6-CPU Scaling

If the cluster had 3 nodes with 2 CPUs each (6 CPUs total), I would keep the parallelism at 4. Increasing parallelism to 6 would allow six shards to start in the first batch, but with 8 total completions, the remaining two shards would still require a second batch. Therefore, increasing parallelism would not eliminate the need for two execution rounds, while parallelism 4 already fully utilizes the original 4-CPU configuration.

Question 4: API Deployment & Self-Healing

Deployment and Self-Healing Mechanism

The spam-detection API was deployed to the Minikube cluster utilizing a `Deployment` manifest configured with 2 replicas and a `ClusterIP` service.

To test fault tolerance, a running pod was intentionally deleted. Kubernetes immediately detected the discrepancy and provisioned a replacement. This self-healing capability is driven by the **ReplicaSet controller**—a background control loop that continuously compares the cluster's current state against the desired state declared in the manifest. If a pod crashes or is terminated, the ReplicaSet instantly schedules a new pod to strictly maintain the declared replica count, ensuring continuous availability.

**Note: Visual evidence of the self-healing mechanism (`q4-self-healing.png`) can be referenced in the GitHub repository.*

Kubernetes Jobs vs. Deployments

While both constructs manage pods, they serve entirely distinct operational paradigms:

- **Deployments:** Designed for continuous, long-running services (e.g., web servers, APIs). If a pod terminates, the deployment controller assumes it is a failure and immediately restarts it to maintain persistent uptime.
- **Jobs:** Designed for finite, batch-processing tasks (e.g., data migrations, the shard validation scripts from Question 3). A Job ensures that a pod executes its specific task and successfully terminates. Once the exit code is successful, the Job is considered complete and the pod is not restarted.

Zero-Downtime Rolling Update

To demonstrate a rolling deployment strategy, the API logic was updated to return a version string (`v2`) and packaged into a new Docker image. The deployment was updated via `kubect1 set image`. Kubernetes executed a zero-downtime rollout by incrementally provisioning `v2` pods and verifying their readiness probes before terminating the old `v1` pods.

**Note: Visual evidence of the rolling update progress and revision history (`q4-rolling-update.png`) can be referenced in the GitHub repository. And also any other proofs required are clearly shown in the screen recording.*